Mini Juego "Piedra, Papel o Tijera" con Persistencia de Datos

Actividad 7 - Laboratorio de Programacion (LPR)

E.E.S.T. N° 1 "Eduardo Ader" - Vicente Lopez

Autor: Malek Cheheid - Grupo A

Docente: Yamil Ganduglia

Fecha: 16 de octubre de 2026

Repositorio: https://github.com/malekcheheid-art/piedrapapelotijera.git

Resumen

El presente informe describe el desarrollo de un mini juego de Piedra, Papel o Tijera contra la inteligencia artificial de la consola, implementado en C++ con persistencia de datos en archivo de texto. Se utiliza una estructura struct para agrupar los datos de cada partida y las bibliotecas ofstream e ifstream para guardar y recuperar el historial.

Objetivos

- Modelar datos heterogeneos mediante struct.
- Persistir resultados en disco con ofstream en modo ios::app.
- Recuperar historial con ifstream.
- Aplicar programacion modular.

Descripcion del sistema

El programa presenta un menu con tres opciones: jugar, ver historial y salir. Al jugar, se solicita el nombre del jugador y su eleccion, se genera la eleccion de la IA de forma aleatoria y se determina el resultado. La partida se guarda en src/historial_partidas.txt con formato CSV.

Persistencia en disco

La funcion guardarEnDisco utiliza ofstream con la bandera ios::app, lo que agrega los registros al final sin borrar los anteriores. La funcion leerHistorialDisco utiliza ifstream para leer todos los registros y mostrarlos en pantalla. Esto demuestra la diferencia entre la memoria RAM (volatil) y el almacenamiento secundario (no volatil).

Estructura de datos

La estructura Partida contiene los campos idPartida, jugador, eleccionJugador, eleccionIA y resultado. Esta agrupacion simplifica el paso de datos entre funciones frente al uso de variables

sueltas.

Cuestionario tecnico

1. RAM vs. Disco: al cerrar la consola se pierden las variables en RAM, pero el archivo en disco conserva las partidas, por lo que la opcion 2 sigue mostrando el historial.

2. Uso de ios::app: sin esta bandera, ofstream sobrescribe el archivo y se pierden las partidas anteriores.

3. cin.ignore(): limpia el caracter Enter pendiente en el bufer antes de usar getline().

4. struct: agrupa variables relacionadas, facilitando su manejo y paso entre funciones.

5. Aplicacion al P.I.A.: se podrian guardar logs de sensores, usuarios, stock o turnos en archivos de texto.

Evidencias

Se incluyen capturas de pantalla de una partida jugada y del archivo historial_partidas.txt mostrando los registros persistidos.

Conclusion

El proyecto cumple con los objetivos de la actividad: uso de struct y persistencia en disco con fstream, manteniendo un historial de partidas entre ejecuciones.

Referencias

- GeeksforGeeks - Structs y estructuras de datos en C++
- Microsoft Learn - Flujos de archivos en C++
- cppreference - Modos de apertura de archivos